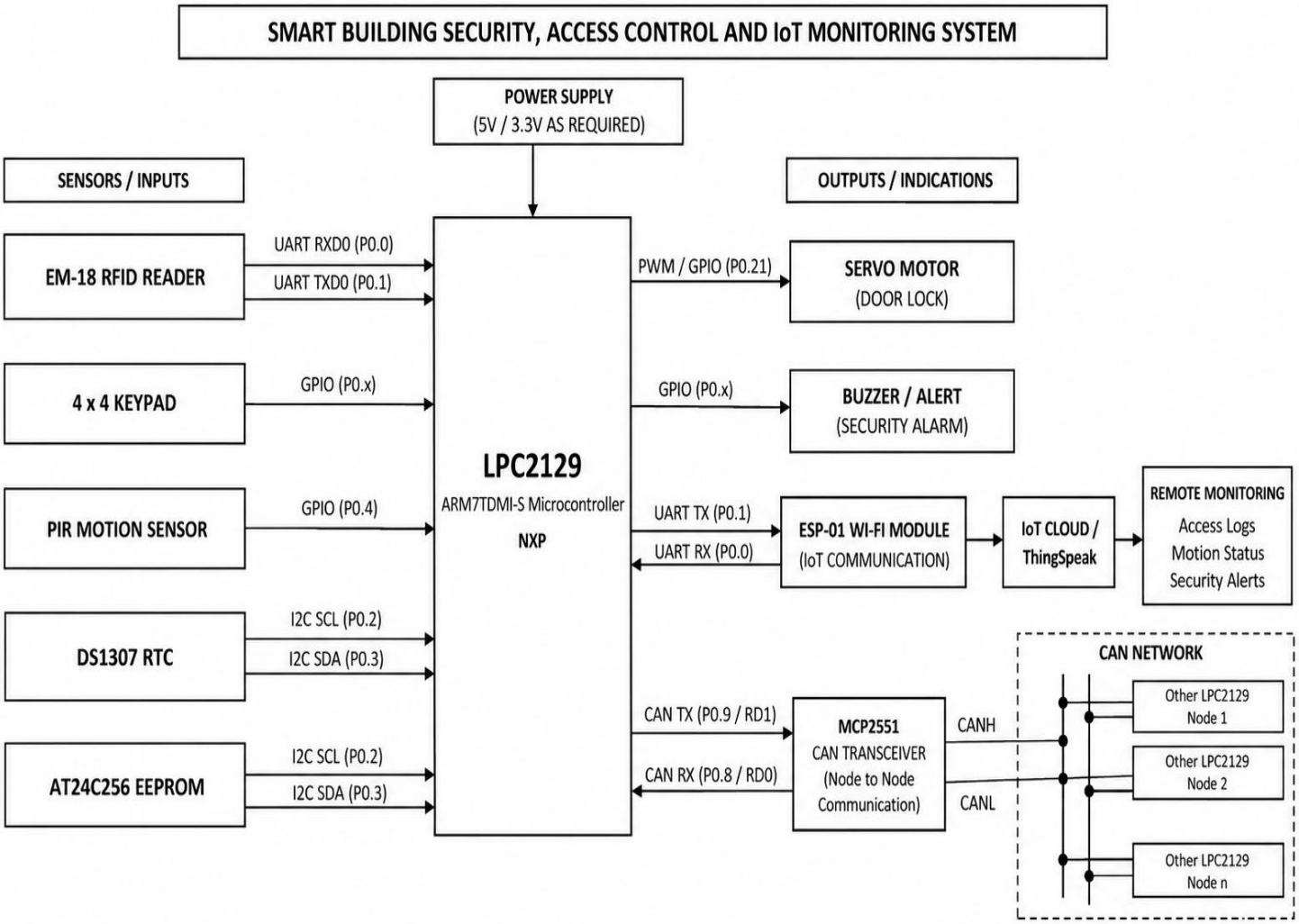
Project Title: Smart Building Security, Access Control and IOT Monitoring System

ID	PROJECT TITLE	GROUP
V25BE9P1 V25BE9P2 V25BE9P3 V25BE9P4	**Smart Building Security, Access Control and IOT Monitoring System**	GROUP-10

Block Diagram



Source Code:

1) header.h

```
#include<lpc21xx.h>
#include<string.h>
typedef unsigned char u8;
//timer functions declerations
extern void delay_sec(unsigned int sec);
extern void delay_ms(unsigned int ms);
//lcd functions declerations
extern void lcd_data(unsigned char data);
extern void lcd_cmd(unsigned char cmd);
extern void lcd_init(void);
extern void lcd_integer(int num);
extern void lcd_string(char * ptr);
extern void bcd_lcd(unsigned char value);
//uart0 functions declerations
extern void uart0_init(unsigned int baud);
extern void uart0_tx(unsigned char data);
extern unsigned char uart0_rx(void);
extern void uart0_tx_string(unsigned char * ptr);
extern void uart0_integer(int num);
extern void uart0_rx_string(unsigned char *ptr ,int len);
extern void bcd_uart0(unsigned char value);
//uart1 functions declerations
extern void uart1_init(unsigned int baud);
extern unsigned char uart1_rx(void);
//RFID declerations
extern int rfid_read(void);
extern int rfid_verify(char * s);
//KEY PAD declerations
extern unsigned char key_read(void);
extern void keypad_init(void);
extern int keypad_verify(void);
typedef struct RTC
{
    unsigned char sec;
    unsigned char min;
```

```
    unsigned char hr;
    unsigned char day;
    unsigned char date;
    unsigned char month;
    unsigned char year;
}RTC;


//i2c header files
extern void i2c_init(void);
extern void i2c_write(u8 sa,u8 mr ,u8 data);
extern unsigned char i2c_read(u8 sa,u8 mr);
extern void set_eeprom(void);
extern void set_rtc(void);
extern void read_rtc(void);
extern void print_RTC(char *ptr);
extern void print_rtc(void);
extern void success_data(void);
extern void failure_data(void);
extern void printSuccessData(void);
void printFailureData(void);


//servo declerations
extern void servo_init(void);
extern void servo_0(void);
extern void servo_90(void);
extern void servo_180(void);
extern void gate_operation(void);
```

2) main.c

```
#include"header.h"

#define PIR (IOPIN0>>4)
#define GREEN_LED (1<<5)
#define RED_LED (1<<6)
#define BUZZ (1<<7)
int ext0=0;    //for external interrupt
int ext1=0;    //for external interrupt
int ext2=0;    //for external interrupt
int main()
{
    int c=0,i;
    char ch;
    int f=0;
    lcd_init();
    uart0_init(9600);
    //uart1_init(9600);
    i2c_init();
    //keypad_init();
    servo_init();
    config_eint0();
    config_eint1();
    IODIR0 |=GREEN_LED|RED_LED|BUZZ;
    IOSET0 =GREEN_LED|RED_LED|BUZZ;
    uart0_tx_string("\r\n-------system started--------\r\n");
    lcd_string("SMART BUILDING");
    uart0_tx_string("STORING USER PREDEFINED DATA\r\n");
    set_eeprom();
    uart0_tx_string("STORING compleated\r\n");
    uart0_tx_string("Data in the eeprom\r\n");
    for(i=0;i<12;i++)
    {
        ch=i2c_read(0xa1,i);
        uart0_tx(ch);
    }
    uart0_tx_string("\r\n");
    set_rtc();
    uart0_tx_string("RTC data and time set\r\n");

    read_rtc();
    print_rtc();
    uart0_tx_string("Monitoring satrted\r\n");
    while(1)
    {
        //c=0;
        lcd_cmd(0x01);
        lcd_string("Monitoring...");
        if(((PIR&1)==1) && ext0==0 && ext1==0 && ext2==0)          //when motion detected
```

```
{
    lcd_cmd(0x01);
    lcd_string("Motion Detected");
    uart0_tx_string("Movement detected\r\n");
    delay_ms(10);
    while(c<2)
    {
        if(ext0!=0 || ext1!=0 || ext2!=0 )
            break;
        lcd_cmd(0x01);
        lcd_string("RFID Ready");
        lcd_cmd(0xc0);
        lcd_string("Scan Card");
        uart0_tx_string("RFID is ready\r\n");
        uart0_tx_string("SCAN YOUR CARD...\r\n");
        if(rfid_read())
        {
                if(ext0!=0 || ext1!=0 || ext2!=0 )
                    break;
                uart0_tx_string("\r\nRFID Authentication Successful...\r\n");
                lcd_cmd(0x01);
                lcd_string("Access Granted");
                IOCLR0=GREEN_LED;
                delay_ms(700);
                IOSET0=GREEN_LED;
                f=1;
                goto  send;
        }
        else
        {
            if(ext0!=0 || ext1!=0 || ext2!=0 )
                break;
            uart0_tx_string("\r\nRFID Authentication not Successful...\r\n");
            lcd_cmd(0x01);
            lcd_string("Access Denied");
            IOCLR0=RED_LED|BUZZ;
            delay_ms(700);
            IOSET0=RED_LED|BUZZ;
            c++;
        }
    }
    c=0;
    lcd_cmd(0x01);
    lcd_string("KEYPAD Ready");
    lcd_cmd(0xc0);
    lcd_string("ENTER PASSWORD");
    uart0_tx_string("KEYPAD is ready...\r\n");
    while(c<2)
    {
        if(ext0!=0 || ext1!=0 || ext2!=0 )
            break;
```

```
            uart0_tx_string("ENTER YOUR PASSWORD...\r\n");
            if(keypad_verify())
            {
                        if(ext0!=0 || ext1!=0 || ext2!=0 )
                              break;
                        uart0_tx_string("PASSWORD Authentication Successful...\r\n");
                        lcd_cmd(0x01);
                        lcd_string("Access Granted");
                        IOCLR0=GREEN_LED;
                        delay_ms(700);
                        IOSET0=GREEN_LED;
                        f=1;
                        goto  send;
            }
            else
            {
                        if(ext0!=0 || ext1!=0 || ext2!=0 )
                              break;
                        uart0_tx_string("PASSWORD Authentication NOT Successful...\r\n");
                        lcd_cmd(0x01);
                        lcd_string("Access Denied");
                        IOCLR0=RED_LED|BUZZ;
                        delay_ms(700);
                        IOSET0=RED_LED|BUZZ;
                        c++;
            }
      }
      send:
      if(f==1 && ext0==0 && ext1==0 && ext2==0)
      {
            c=0;
            f=0;
            success_data();
            uart0_tx_string("Data saved successfully\r\n");
            IOCLR0=GREEN_LED;
            delay_ms(700);
            IOSET0=GREEN_LED;
            gate_operation();
      }
      else if(f==0 && ext0==0 && ext1==0 && ext2==0)
      {
            c=0;
            f=0;
            failure_data();
            uart0_tx_string("Too many attempts security ALERT\r\n");
            IOCLR0=RED_LED|BUZZ;
            delay_ms(700);
            IOSET0=RED_LED|BUZZ;
            IOCLR0=BUZZ;
      }
}
```

```
	else if(ext0==1)
	{
		uart0_tx_string("MANUAL DOOR MODE\r\n");
		lcd_cmd(0x01);
		lcd_string("MANUAL OPEN");
		//open door
		servo_180();
		uart0_tx_string("MANUAL DOOR OPEN\r\n");
		delay_ms(500);
		uart0_tx_string("MANUAL DOOR CLOSE\r\n");
		ext0=0;
	}
	else if(ext1==1)
	{
		uart0_tx_string("Security Alarm Reset\r\n");
		//buzzer reset
		IOSET0=BUZZ;
		ext1=0;
	}
	else if(ext2==1)
	{
		uart0_tx_string("Emergency Mode Activated\r\n");
		//open door
	}
}
```

3) Servo.c

```
#include<lpc21xx.h>
#include"header.h"
//pin 21 for the servo motor change  it based
void servo_init(void)
{
	PINSEL1 |=(1<<10);
	PWMPR=14;
	PWMMR0=20000;
	PWMMR5=1500;
	PWMMCR=1<<1;
	PWMPCR=1<<13;   //PWM5 output enable
	PWMLER=(1<<0)|(1<<5);
	PWMTCR=(1<<0)|(1<<3);
}
void servo_0(void)
{
	PWMMR5=1000;
	PWMLER=(1<<4);
}
void servo_90(void)
{
	PWMMR5=1500;
	PWMLER=(1<<4);
}
void servo_180(void)
{
	PWMMR5=2000;
	PWMLER=(1<<4);
}
void gate_operation(void)
{
	servo_180();
	servo_180();
	delay_ms(100);
	uart0_tx_string("\r\nGATE OPEN\r\n");
	delay_sec(3); //how much time the gate should be open
	uart0_tx_string("\r\nGATE CLOCSE\r\n");
	servo_0();
	servo_0();
	delay_ms(100);
}
```

4) delay.c

```
#include<lpc21xx.h>
#include"header.h"
void delay_sec(unsigned int sec){
T0PR=14999999;
T0PC=0;
T0TC=0;
T0TCR=1;
while(T0TC<sec);
T0TCR=0;
}
void delay_ms(unsigned int ms){
T0PR=14999;
T0PC=0;
T0TC=0;
T0TCR=1;
while(T0TC<ms);
T0TCR=0;
}
```

5) Rfid.c

```
 #include<lpc21xx.h>
#include"header.h"
#include<string.h>

int rfid_read(void)
{
      unsigned char rec_id[13];
      //uart0_tx_string("\r\nrfid initialised..");
      uart0_rx_string(rec_id,12);
     uart0_tx_string("\r\n");
      uart0_tx_string(rec_id);
      if(rfid_verify(rec_id))
            return 1;
      else
            return 0;
}

int rfid_verify(char * s)
{
    unsigned char stor_id[13]="4900C9C08FCF";
     //read from the eeprom and compare it with above received
     if(strcmp(s,stor_id)==0)
          return 1;
     else
           return 0;
}
```

6) I2c_driver.c

```
#include<lpc21xx.h>
#include"header.h"

typedef unsigned char u8;

void i2c_init(void)
{
	PINSEL0 |=0x50;
	I2CONSET=(1<<6);
	I2SCLH=75;
	I2SCLL=75;
}

#define SI ((I2CONSET>>3)&1)

void i2c_write(u8 sa,u8 mr ,u8 data)
{
	I2CONSET=(1<<5);
	I2CONCLR=(1<<3);
	while(SI==0);
	I2CONCLR=(1<<5);

	//SA + W + ACk
	I2DAT=sa;
	I2CONCLR=(1<<3);
	while(SI==0);
	if(I2STAT==0x20)
	{
		uart0_tx_string("ERR : SA + W\r\n");
		goto exit;
	}

	//MR + ACk
	I2DAT=mr;
	I2CONCLR=(1<<3);
	while(SI==0);
	if(I2STAT==0x30)
	{
		uart0_tx_string("ERR :Memory addr \r\n");
		goto exit;
	}

	//data +ack
	I2DAT=data;
	I2CONCLR=(1<<3);
	while(SI==0);
	if(I2STAT==0x30)
```

```
	{
		uart0_tx_string("ERR : ");
		goto exit;
	}

	//stop condition
	exit:
		I2CONSET=(1<<4);
		I2CONCLR=(1<<3);

}


unsigned char i2c_read(u8 sa,u8 mr)
{
	u8 temp;
	//dummy write
	//start condition
	I2CONSET=(1<<5);
	I2CONCLR=(1<<3);
	while(SI==0);
	I2CONCLR=(1<<5);

	//SA + W + ACk
	I2DAT=sa^1;
	I2CONCLR=(1<<3);
	while(SI==0);
	if(I2STAT==0x20)
	{
		uart0_tx_string("ERR : SA + W\r\n");
		goto exit;
	}

	//MR + ACk
	I2DAT=mr;
	I2CONCLR=(1<<3);
	while(SI==0);
	if(I2STAT==0x30)
	{
		uart0_tx_string("ERR :Memory addr \r\n");
		goto exit;
	}

	//restart
	//start condition
	I2CONSET=(1<<5);
	I2CONCLR=(1<<3);
	while(SI==0);
	I2CONCLR=(1<<5);

	//SA + R + ACk
```

```
	I2DAT=sa;
	I2CONCLR=(1<<3);
	while(SI==0);
	if(I2STAT==0x48)
	{
		uart0_tx_string("ERR : SA + W\r\n");
		goto exit;
	}

	I2CONCLR=(1<<3);
	while(SI==0);
	temp=I2DAT;

	exit:
		I2CONSET=(1<<4);
		I2CONCLR=(1<<3);
	return temp;
}


void set_eeprom(void)
{
	int i,ma;
	char s[13]="4900C9C08FCF";
	char pass1[5]="1234",pass2[5]="1122";

	/* RFID saved at page 0 (SA->0xA0,MA->0,data)
		total 12 bytes
	*/
	for(i=0;i<12;i++)
	{
		i2c_write(0xA0,i,s[i]);
		delay_ms(20);
	}

	/* key pad pass saved at page 0 (SA->0xA0,MA->126,data)(126 to 130)
		total 4 bytes
	*/
	ma=126;
	for(i=0;i<4;i++)
	{
		  i2c_write(0xA0,ma,pass1[i]);
		  delay_ms(20);
		  ma++;
	}
	/* key pad pass saved at page 0 (SA->0xA0,MA->130,data)(130 to 134)
		total 4 bytes
	*/
	for(i=0;i<4;i++)
	{
		  i2c_write(0xA0,ma,pass2[i]);
```

```
            delay_ms(20);
            ma++;
      }

}


//for setting time to the rtc
void set_rtc(void)
{
      i2c_write(0xd0,0x00,0x22); //sec
      i2c_write(0xd0,0x01,0x12);     //min
      i2c_write(0xd0,0x02,0x04);  //hr
      i2c_write(0xd0,0x04,0x13);  //date
      i2c_write(0xd0,0x05,0x10);  //month
      i2c_write(0xd0,0x06,0x26);  //year
}


//reading time from the rtc module
RTC t1;
void read_rtc(void)
{
        t1.sec=i2c_read(0xD1,0x00);
        //t1.sec=t1.sec&0x7f;
        t1.min=i2c_read(0xD1,0x01);
        t1.hr=i2c_read(0xD1,0x02);
        //t1.hr=t1.hr&0x3f;
        t1.day=i2c_read(0xD1,0x03);
        t1.date=i2c_read(0xD1,0x04);
        t1.month=i2c_read(0xD1,0x05);
        t1.year=i2c_read(0xD1,0x06);
}

//printing the rtc data
void print_RTC(char *ptr)
{
      unsigned char sec;
      unsigned char min;
      unsigned char hr;

      unsigned char date;
      unsigned char month;
      unsigned char year;

      sec=ptr[0] & 0x7f; ///?
      min=ptr[1] & 0x7f;
      hr=ptr[2] & 0x3f;

      date=ptr[4];
      month=ptr[5];
```

```
	year=ptr[6];

	uart0_tx_string("\r\nDate : ");
	bcd_uart0(date);
	uart0_tx('/');
	bcd_uart0(month);
	uart0_tx('/');
	bcd_uart0(year);

	uart0_tx_string("\r\nTime : ");
	bcd_uart0(hr);
	uart0_tx(':');
	bcd_uart0(min);
	uart0_tx(':');
	bcd_uart0(sec);

	lcd_cmd(0x01);
	lcd_cmd(0x80);
	lcd_string("D:");
	bcd_lcd(date);
	lcd_data(':');
	bcd_lcd(month);
	lcd_data(':');
	bcd_lcd(year);

	lcd_cmd(0xC0);
	lcd_string("T:");
	bcd_lcd(hr);
	lcd_data(':');
	bcd_lcd(min);
	lcd_data(':');
	bcd_lcd(sec);
	 return ;
}

void print_rtc(void)
{
	char rt[8];
	read_rtc();
	rt[0]=t1.sec;
	rt[1]=t1.min;
	rt[2]=t1.hr;
	rt[3]=t1.day;
	rt[4]=t1.date;
	rt[5]=t1.month;
	rt[6]=t1.year;
	print_RTC(rt);
}

//save authentication data
int sda=256;
```

```
void success_data(void)
{
	unsigned char data[8];
	int i;
	data[0]=t1.sec;
	data[1]=t1.min;
	data[2]=t1.hr;
	data[3]=t1.day;
	data[4]=t1.date;
	data[5]=t1.month;
	data[6]=t1.year;

	for(i=0;i<7;i++)
	{
		i2c_write(0xA2,sda,data[i]);
		delay_ms(20);
		sda++;

	}

}

//failure dara store
int fda=512;
void failure_data(void)
{
  unsigned char data[8];
	int i;
	data[0]=t1.sec;
	data[1]=t1.min;
	data[2]=t1.hr;
	data[3]=t1.day;
	data[4]=t1.date;
	data[5]=t1.month;
	data[6]=t1.year;

	for(i=0;i<7;i++)
	{
		i2c_write(0xA4,fda,data[i]);
		delay_ms(20);
		fda++;

	}
}

void printSuccessData(void)
{
	unsigned char c[8];
	unsigned int read_addr=256;
	int i;
	uart0_tx_string("\r\n******SUCCESS AUTHENTICATION LOG*******\r\n");
```

```
    while(read_addr<sda)
    {
        for(i=0;i<7;i++)
        {
            c[i]=i2c_read(0xA3,read_addr);
            read_addr++;
        }
        print_RTC(c);
        delay_ms(100);
    }
}

void printFailureData(void)
{
  unsigned char c[8];
    unsigned int read_addr=512;
    int i;
    uart0_tx_string("\r\n******FAILURE AUTHENTICATION LOG*******\r\n");
    while(read_addr<fda)
    {
        for(i=0;i<7;i++)
        {
            c[i]=i2c_read(0xA3,read_addr);
            read_addr++;
        }
        print_RTC(c);
       delay_ms(100);
}
}
```

7) Interrupt.c

```
#include"header.h"

extern int ext0;
extern int ext1;
extern int ext2;

void EINT0_handler(void)__irq
{
	ext0=ext0^1;
	EXTINT=1;
	VICVectAddr=0;
}

void EINT1_handler(void)__irq
{
	ext1=ext1^1;
	EXTINT|=1<<3;
	VICVectAddr=0;
}


void config_eint0(void)
{
	PINSEL1|=1;
	EXTMODE=1;
	EXTPOLAR=0;
	VICIntSelect=0;
	VICVectCntl2=14|(1<<5);
	VICVectAddr2=(unsigned int)EINT0_handler;
	VICIntEnable=(1<<14);
}

void config_eint1(void)
{
	PINSEL1|=0x0300;
	EXTMODE |=1<<3;
	EXTPOLAR =0;
	VICIntSelect=0;
	VICVectCntl3=17|(1<<5);
	VICVectAddr3=(unsigned int)EINT1_handler;
	VICIntEnable=(1<<17);
}
```

8) Keypad_driver.c

```
#include<lpc21xx.h>
#include"header.h"

#define R1 (1<<8)
#define R2 (1<<9)
#define R3 (1<<10)
#define R4 (1<<11)

#define C1 (1<<12)
#define C2 (1<<13)
#define C3 (1<<14)
#define C4 (1<<15)


void keypad_init(void)
{
    IODIR0 |=(R1|R2|R3|R4);       //OUT DIR
    IODIR0 &=~(C1|C2|C3|C4);   //INPUT DIR
}

char keypad_read(void)
{
            while(1)
            {
                IOSET0 =R2|R3|R4;
                IOCLR0 =R1;

                if((IOPIN0&C1)==0)
                {
                    delay_ms(20);
                    while((IOPIN0&C1)==0);
                    return '1';
                }
                if((IOPIN0&C2)==0)
                {
                    delay_ms(20);
                    while((IOPIN0&C2)==0);
                    return '2';
                }
                if((IOPIN0&C3)==0)
                {
                    delay_ms(20);
                    while((IOPIN0&C3)==0);
                    return '3';
                }
                if((IOPIN0&C4)==0)
```

```
{
	delay_ms(20);
	while((IOPIN0&C4)==0);
	return 'A';
}

IOSET0 =R1|R3|R4;
IOCLR0 =R2;

if((IOPIN0&C1)==0)
{
	delay_ms(20);
	while((IOPIN0&C1)==0);
	return '4';
}
if((IOPIN0&C2)==0)
{
	delay_ms(20);
	while((IOPIN0&C2)==0);
	return '5';
}
if((IOPIN0&C3)==0)
{
	delay_ms(20);
	while((IOPIN0&C3)==0);
	return '6';
}
if((IOPIN0&C4)==0)
{
	delay_ms(20);
	while((IOPIN0&C4)==0);
	return 'B';
}

IOSET0 =R1|R2|R4;
IOCLR0 =R3;

if((IOPIN0&C1)==0)
{
	delay_ms(20);
	while((IOPIN0&C1)==0);
	return '7';
}
if((IOPIN0&C2)==0)
{
	delay_ms(20);
	while((IOPIN0&C2)==0);
	return '8';
}
if((IOPIN0&C3)==0)
{
```

```
                        delay_ms(20);
                        while((IOPIN0&C3)==0);
                        return '9';
                    }
                    if((IOPIN0&C4)==0)
                    {
                        delay_ms(20);
                        while((IOPIN0&C4)==0);
                        return 'C';
                    }

                    IOSET0 =R1|R2|R3;
                    IOCLR0 =R4;

                    if((IOPIN0&C1)==0)
                    {
                        delay_ms(20);
                        while((IOPIN0&C1)==0);
                        return '*';
                    }
                    if((IOPIN0&C2)==0)
                    {
                        delay_ms(20);
                        while((IOPIN0&C2)==0);
                        return '0';
                    }
                    if((IOPIN0&C3)==0)
                    {
                        delay_ms(20);
                        while((IOPIN0&C3)==0);
                        return '#';
                    }
                    if((IOPIN0&C4)==0)
                    {
                        delay_ms(20);
                        while((IOPIN0&C4)==0);
                        return 'D';
                    }
                }
}


int keypad_verify(void)
{
        char real_pass[5];
        //read saved pass word through eeprom
        char a[5];
        int i;
        for(i=0;i<4;i++)
        {
            real_pass[i]=i2c_read(0xa1,i);
```

```
        }
        for(i=0;i<4;i++)
        {
            //a[i]=keypad_read();
            //lcd_data('*');
            a[i]=uart0_rx();
            if(a[i]=='\r')
                break;
            uart0_tx(a[i]);
        }
        a[i]='\0';
        //uart0_rx_string(a,5);
        if(strcmp(a,real_pass)==0)
            return 1;
        else
            return 0;
}
```

9) Lcd_4bit_driver.c

```
#include<lpc21xx.h>
#include"header.h"

void lcd_data(unsigned char data)
{
    unsigned int result;
    //higher nibble
    IOCLR1=0xfe<<16;
    result=(data&0xf0)<<16;
    IOSET1=result;
    IOSET1=1<<17;
    IOCLR1=1<<18;
    IOSET1=1<<19;
    delay_ms(2);
    IOCLR1=1<<19;

    //lower nibble
    IOCLR1=0xfe<<16;
    result=(data&0x0f)<<20;
    IOSET1=result;
    IOSET1=1<<17;
    IOCLR1=1<<18;
    IOSET1=1<<19;
    delay_ms(2);
    IOCLR1=1<<19;
}

void lcd_cmd(unsigned char cmd)
{
    unsigned int result;
```

```
	//higher nibble
	IOCLR1=0xfe<<16;
	result=(cmd&0xf0)<<16;
	IOSET1=result;
	IOCLR1=1<<17;
	IOCLR1=1<<18;
	IOSET1=1<<19;
	delay_ms(2);
	IOCLR1=1<<9;

	//lower nibble
	IOCLR1=0xfe<<16;
	result=(cmd&0x0f)<<20;
	IOSET1=result;
	IOCLR1=1<<17;
	IOCLR1=1<<18;
	IOSET1=1<<19;
	delay_ms(2);
	IOCLR1=1<<19;
}

void lcd_init(void)
{
	IODIR1=0xfe<<16;
	PINSEL2 |=0;
	lcd_cmd(0x02);
	lcd_cmd(0x28);
	lcd_cmd(0x0e);
	lcd_cmd(0x01);

}

void lcd_string(char * ptr)
{
	while(*ptr)
	{
		lcd_data(*ptr);
		ptr++;
	}
}

void lcd_integer(int num)
{
	int i=0;
	char a[10];
	if(num==0)
	{
		lcd_data('0');
		return ;
	}
	if(num<0)
```

```
    {
        num=-num;
        lcd_data('-');
    }

    while(num)
    {
        a[i]=num%10 +48;
        i++;
        num=num/10;
    }

    for(i=i-1;i>=0;i--)
    {
        lcd_data(a[i]);
    }
}

void bcd_lcd(unsigned char value)
{
    lcd_data((value>>4)+'0');
    lcd_data((value&0x0f)+'0');
}
```

10) Uart0_driver.c

```
#include<lpc21xx.h>
#include"header.h"

void uart0_init(unsigned int baud)
{
    int result,pclk;
    if(VPBDIV==0)
        pclk=15000000;
    else if(VPBDIV==1)
        pclk=60000000;
    else if(VPBDIV==2)
        pclk=30000000;
    PINSEL0 |=0x05;

    result=pclk/(16*baud);
    U0LCR=0x83;
    U0DLL=result&0xff;
    U0DLM=(result>>8)&0xff;
    U0LCR=0x03;

}

#define THRE ((U0LSR>>5)&1)
void uart0_tx(unsigned char data)
{
    U0THR=data;
```

```
	while(THRE==0);
}

#define RDR (U0LSR&1)
unsigned char uart0_rx(void)
{
	while(RDR==0);
	return U0RBR;
}

//uart0 string
void uart0_tx_string(unsigned char * ptr)
{
	while(*ptr)
	{
		uart0_tx(*ptr);
		ptr++;
	}
}

void uart0_integer(int num)
{
	int i=0;
	char a[10];
	if(num==0)
	{
		uart0_tx('0');
		return;
	}
	if(num<0)
	{
		num=-num;
		uart0_tx('-');
	}

	while(num)
	{
		a[i]=num%10 +48;
		i++;
		num=num/10;
	}

	for(i=i-1;i>=0;i--)
		uart0_tx(a[i]);
}

extern int ext0;
extern int ext1;
extern int ext2;
void uart0_rx_string(unsigned char *ptr ,int len)
{
	int i;
	for(i=0;i<len;i++)
	{
	// && ext0!=1 && ext1!=1 && ext2!=1
		while(RDR==0 && ext0!=1 && ext1!=1 && ext2!=1);
```

```
            ptr[i]=U0RBR;
            if(ptr[i]=='\r')
                  break;
            if(ext0==1 || ext1==1 || ext2==1)
                  break;
      }
      ptr[i]='\0';
}

void bcd_uart0(unsigned char value)
{
      uart0_tx((value>>4)+'0');
      uart0_tx((value&0x0f)+'0');
}
```

11) Uart1_driver.c

```
#include"header.h"
void uart1_init(unsigned int baud)
{
        int result,pclk;
       if(VPBDIV==0)
            pclk=15000000;
       else if(VPBDIV==1)
            pclk=60000000;
       else if(VPBDIV==2)
            pclk=30000000;
       PINSEL0 |=0x050000;

       result=pclk/(16*baud);
       U1LCR=0x83;
       U1DLL=result&0xff;
       U1DLM=(result>>8)&0xff;
       U1LCR=0x03;

}

#define RDR (U1LSR&1)
unsigned char uart1_rx(void)
{
       while(RDR==0);
      return U1RBR;
}
```